B4 Polars and DuckDB

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B4 Polars and DuckDB 3
B4-L1 Polars expressions and lazy frames 3
B4-L2 DuckDB, SQL over files and DataFrames 7
B4-L3 Choosing pandas, Polars, DuckDB or Spark 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B4 Polars and DuckDB

Roadmap: Prerequisite for all months. Tools for larger local datasets.

B4-L1 Polars expressions and lazy frames

Concept

Polars is a DataFrame library written in Rust. It runs on all your cores and has no row index. You describe work with **expressions**: `pl.col("weight").sum()` is a recipe, not a result. Polars runs it inside a context.

```
import polars as pl

shares = (
    df.group_by("wave", "brand")
    .agg(pl.col("weight").sum().alias("brand_weight"))
    .with_columns(total=pl.col("brand_weight").sum().over("wave"))
)
```

- `select` returns only the expressions you list. `with_columns` adds or replaces columns. `filter` keeps rows. `group_by(...).agg(...)` aggregates.
- `.over("wave")` computes a group total and broadcasts it back to each row, like a window function in SQL.
- `pl.when(...).then(...).otherwise(...)` replaces `np.where`.

Polars has two modes. A `DataFrame` runs each step at once. A `LazyFrame`, from `df.lazy()` or `pl.scan_parquet(path)`, records the steps and runs nothing until `.collect()`. Before running, the optimiser rewrites the plan. It pushes filters down to the scan, so rows you drop are never read, and reads only the columns you use. `lf.explain()` prints the plan.

Two behaviours to know (Polars 1.33):

- `sum()` **of a group with only nulls returns** `0`, not null. Check `null_count()` before you trust a total.
- `mean()` **skips nulls.** `sum() / pl.len()` counts null rows in the denominator and gives a different answer.

Use lazy mode for files. Scan, filter, select, aggregate, then collect once at the end.

Why it matters

A weekly job reads a 9 GB Parquet extract with `pl.read_parquet`, then filters to the current wave. Memory peaks at 14 GB and the job fails on a 16 GB runner on the week the extract grows.

`pl.scan_parquet(path).filter(pl.col("wave") == wave).select(cols).collect()` lets Polars skip other waves' row groups and unused columns at read time, so memory scales with one wave, not the whole file.

Worked example: weak code

```
1 df = pl.read_parquet("responses.parquet")
2 df = df.filter(pl.col("wave") == "2026-09")
3 totals = df.group_by("brand").agg(pl.col("weight").sum())
4 totals = totals.with_columns(mean_w=pl.col("weight") / df.height)
```

1. **Line 1.** Eager read: every wave and every column loads before the filter.
2. **Line 4.** Divides a brand total by the whole wave's row count: not the brand mean, and null rows count.

Worked example: strong code

```
1  import polars as pl
2
3
4  def brand_weights(path: str, wave: str) -> pl.DataFrame:
5      """Weighted totals and means per brand for one wave. Grain: one row per brand."""
6      return (
1          pl.scan_parquet(path)
2          .filter(pl.col("wave") == wave)
9          .group_by("brand")
10         .agg(
11             total=pl.col("weight").sum(),
12             mean_w=pl.col("weight").mean(),
3              missing=pl.col("weight").null_count(),
14         )
15         .sort("brand")
4          .collect()
17     )
```

1. **Line 7.** Lazy scan: nothing is read yet.
2. **Line 8.** Pushed down to the scan: other waves are skipped at read time.
3. **Line 13.** Reports nulls, because an all-null group sums to 0.
4. **Line 16.** One collect at the end runs the optimised plan.

Scan lazily, filter and select early, aggregate with expressions, and collect once.

Recap card

- **Polars eager vs lazy** DataFrame runs each step at once. LazyFrame (scan_*, .lazy()) builds a plan and runs on collect().
- **Group total on every row in Polars** pl.col("x").sum().over("group").
- **Polars sum() of an all-null group** 0, not null. Report null_count() with totals.
- **Mean that skips nulls in Polars** pl.col("x").mean(). sum() / pl.len() counts null rows.

Practice

Exercise 1. Multiple choice (level 1)

`lf = pl.scan_parquet("big.parquet").filter(pl.col("wave") == "w9")`. What has happened after this line runs?

- a) The whole file is in memory, filtered to wave w9.
- b) Nothing is read yet. lf holds a query plan.
- c) Only the w9 rows are in memory.
- d) Polars wrote a filtered copy of the file.

Why?

- r1) A LazyFrame records steps and runs them only on collect().
- r2) Parquet files cannot be read without DuckDB.

Exercise 2. Predict the output (level 2)

What does this print on Polars 1.33?

What you need to know.

`series.to_list()` returns the values of a Polars Series as a plain Python list. `out["w"]` picks one column of a Polars DataFrame as a Series.

```
pl.DataFrame({"n": [1, 2]})["n"].to_list()   # [1, 2]
```

```
import polars as pl

df = pl.DataFrame({"wave": ["w1", "w1", "w2"], "w": [1.0, 2.0, None]})
out = df.group_by("wave").agg(pl.col("w").sum()).sort("wave")
print(out["w"].to_list())
```

Exercise 3. Fill the gap (level 2)

Add a `share` column with each brand's weight over its wave's total, then run the lazy plan.

```
out = (
```

```
    lf.with_columns(share=pl.col("weight") /
    pl.col("weight").sum().over("wave"))
    .pipe(top_n, 2)
)
```

Exercise 4. Write the function (level 3)

Write `brand_share(df: pl.DataFrame) -> pl.DataFrame`. Input columns: `wave`, `brand`, `weight`. Return one row per wave and brand with columns `wave`, `brand`, `share`, where `share` is the brand's weight total over the wave's weight total. When a wave's total is 0, `share` is null. Sort by `wave`, then `brand`.

Your code must pass these checks:

- Each brand's share must be its weight over the wave total, one row per wave and brand.
- A wave with zero total weight must give a null share, not NaN or an error.
- Return exactly wave, brand, share, sorted by wave then brand.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Weighted share of each brand within its wave. Grain: one row per wave
    and brand.
def brand_share(df: pl.DataFrame) -> pl.DataFrame: ...
```

Starter code:

```
import polars as pl


def brand_share(df):
    ...
```

Exercise 5. Spot the bug (level 4)

The mean satisfaction score per region looks too low in regions with many skipped answers. Tap the line with the bug and fix it.

```
import polars as pl


def mean_score(df: pl.DataFrame) -> pl.DataFrame:
    """Mean score per region among respondents who answered."""
    return (
        df.group_by("region")
        .agg(mean_score=pl.col("score").sum() / pl.len())
        .sort("region")
    )
```

Write down the line number of the bug. Then write the fix.

Sources

- Polars sum() of a group with only nulls returns 0. (verified: Run on Polars 1.33.1 and 1.44.2 in this repository.)
- LazyFrame optimisation pushes filters and column selection down to the scan. (verified: Polars user guide, lazy API and optimisations; plan checked with explain() on Polars 1.33.1.)

B4-L2 DuckDB, SQL over files and DataFrames

Concept

DuckDB is an analytical SQL database that runs inside your Python process. There is no server to install. It reads Parquet, CSV and JSON files directly, and it queries pandas and Polars DataFrames that are in scope.

```
import duckdb

con = duckdb.connect()   # in-memory database
rel = con.sql("""
    SELECT wave, brand, sum(weight) AS total
    FROM read_parquet('extracts/*.parquet')
    GROUP BY wave, brand
""")
df = rel.df()    # pandas; .pl() for Polars, .fetchall() for tuples
```

- A glob such as `'extracts/*.parquet'` reads many files as one table. Filters and column lists are pushed down to the reader, as in Polars lazy mode.
- `FROM responses` finds a DataFrame variable named `responses` in the calling scope. `con.register("responses", df)` names it explicitly, which is clearer inside functions.
- Results come back as a relation. Nothing is materialised until you call `.df()`, `.pl()`, `.fetchall()` or `.fetchone()`.

Pass values as parameters, never with f-strings.

```
con.execute("SELECT count(*) FROM responses WHERE wave = ?", [wave])
```

With `f"... WHERE wave = '{wave}'"`, a value that holds a quote changes the query. In this course's own test, the input `w1' or '1'='1` turned a filter for one wave into a count of every row. A `?` placeholder (or `$name` with a dict) sends the value as data, so quotes cannot change the SQL. File paths and table names cannot be parameters. Check those against a list of allowed values.

Create the connection at the edge of your program and pass `con` to functions. The module-level `duckdb.sql()` uses one shared default connection, a global that tests cannot replace.

Why it matters

A dashboard helper built its filter with `f"WHERE region = '{region}'"`. The region list came from a client file. One entry was `Hawke's Bay`. The apostrophe broke the SQL, and the job failed at 06:00 before the client's morning meeting. A second entry with a crafted value could have read other clients' rows.

`con.execute("... WHERE region = ?", [region])` handles `Hawke's Bay` as a plain string.

Worked example: weak code

```
1  import duckdb
2
3
4  def wave_total(wave):
1      q = f"SELECT sum(weight) FROM read_parquet('data/*.parquet') WHERE
       wave = '{wave}'"
2      return duckdb.sql(q).fetchone()[0]
```

1. **Line 5.** Value pasted into SQL: a quote in wave breaks or rewrites the query.
2. **Line 6.** Shared default connection, and None when no rows match.

Worked example: strong code

```
1  import duckdb
2
1  TOTAL_SQL = "SELECT sum(weight) FROM responses WHERE wave = ?"
4
5
2  def wave_total(con: duckdb.DuckDBPyConnection, wave: str) -> float:
7      """Total weight for one wave, 0.0 when the wave has no rows."""
8      row = con.execute(TOTAL_SQL, [wave]).fetchone()
3      if row is None or row[0] is None:
10         return 0.0
11     return float(row[0])
```

1. **Line 3.** The value goes in as a parameter. The SQL text never changes.
2. **Line 6.** The connection is passed in, so a test gives it a small in-memory table.
3. **Line 9.** SQL sum over no rows is NULL. Test with `is None`.

One connection passed in, parameters for values, and a None check on aggregates.

Recap card

- **Query many Parquet files in DuckDB** FROM read_parquet('dir/*.parquet').
- **Pass a value into DuckDB SQL** con.execute("... WHERE x = ?", [value]). Never an f-string.
- **Table names from user input** Placeholders cannot bind identifiers. Check the name against an allowlist.
- **SQL sum over no rows** NULL, returned as None. Check with is None.

Practice

Exercise 1. Multiple choice (level 1)

Which call safely filters by a `region` value read from a client file?

- a) `con.execute(f"... WHERE region = '{region}'")`
- b) `con.execute("... WHERE region = ?", [region])`
- c) `con.execute("... WHERE region = '" + region + "'")`
- d) `con.execute("... WHERE region = '%s'" % region)`

Why?

- r1) The value is sent as data, separate from the SQL text.
- r2) It is shorter.

Exercise 2. Predict the output (level 2)

What does this print? (DuckDB 1.5)

```
import duckdb

con = duckdb.connect()
con.execute("CREATE TABLE t AS SELECT * FROM (VALUES ('w1', 1.0), ('w2', 2.0)) v(wave, weight)")
print(con.execute("SELECT sum(weight) FROM t WHERE wave = 'w9'").fetchone())
```

Exercise 3. Fill the gap (level 2)

Count rows for one wave from all Parquet files in `extracts/`, with the wave as a parameter.

```
con.execute(
    "SELECT count(*) FROM _____(1)('extracts/*.parquet') WHERE wave = _____(2)",
    [wave],
).fetchone()
```

Exercise 4. Write the function (level 3)

Write `region_total(con, region)`. It returns the sum of `weight` in table `responses` for the given `region`, as a `float`. Return `0.0` when no rows match. Pass `region` as a parameter. The checker runs it with regions such as `Hawke's Bay`.

Your code must pass these checks:

- Sum the weights for the region.
- A region with an apostrophe must work. Pass it as a parameter.
- No matching rows must give 0.0, not None.
- A crafted value must match nothing. Pass it as a parameter.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
REGION_TOTAL_SQL = ...

# Total weight for one region.
def region_total(
    con: duckdb.DuckDBPyConnection,
    region: str,
) -> float: ...
```

Starter code:

```python
import duckdb


def region_total(con, region):
    return con.execute(f"SELECT sum(weight) FROM responses WHERE region =
    '{region}'").fetchone()[0]
```

Exercise 5. Architecture design (level 4)

A reporting script lets users pick a table to summarise and a wave to filter on. Choose how each input reaches DuckDB.

Options: A ? parameter, An f-string inside the SQL.

1. The wave value
2. The table name
3. The connection

Sources

- DuckDB binds ? and $name parameters with con.execute(sql, params). (verified: DuckDB Python API documentation; run on DuckDB 1.5.1 and 1.5.5 in this repository.)

- DuckDB queries pandas DataFrames in the calling scope by variable name and reads Parquet globs with read_parquet. (verified: DuckDB Python API documentation; run on DuckDB 1.5.1 and 1.5.5 in this repository.)
- Pasting w1' or '1'='1 into an f-string filter returned every row; the same value as a parameter matched none. (verified: Run on DuckDB 1.5.1 in this repository.)

B4-L3 Choosing pandas, Polars, DuckDB or Spark

Concept

All four tools answer the same questions. Pick by where the data lives and how big it is next to your memory, not by fashion.

A measurement from this course's build: a group-by with sum and mean over 10 million rows and 1,000 groups, best of three runs on a 4-core machine. pandas 3.0.6 took 0.22 s, Polars 1.44 took 0.11 s, DuckDB 1.5 took 0.06 s. At this size every tool is fast enough. The choice matters when data outgrows memory.

Situation	Good default
Data fits comfortably in memory, you need the scikit-learn, statsmodels or plotting ecosystem	pandas
Files larger than you want in memory, heavy transforms, one machine	Polars lazy (`scan_parquet`)
SQL is the clearest way to express it, or you join many files ad hoc	DuckDB
Data already in a lakehouse, or larger than one machine, or a scheduled production job	Spark (Track A)

The tools share the **Apache Arrow** memory format, so you mix them at the boundaries:

```
lazy_result = pl.scan_parquet("extracts/*.parquet").filter(...).group_by(...).agg(...)
summary = lazy_result.collect().to_pandas()     # Polars to pandas for statsmodels
rel = con.sql("SELECT ... FROM read_parquet('extracts/*.parquet')")
top = rel.pl()                                  # DuckDB to Polars
```

Rules for mixing:

- **Reduce first, convert last.** Filter and aggregate in the engine that reads the files. Convert the small result.

- **Watch the index.** Polars has no index. `pl.from_pandas(df)` drops a pandas index unless you pass `include_index=True`, or call `reset_index()` first.
- **Check dtypes after each hop.** Text becomes `str` in pandas 3 and `String` in Polars. Nullable integers need care in both directions.

One project, one primary engine. A second tool at a clear boundary is fine. Four tools in one function is not.

Why it matters

An analyst sets `respondent_id` as the pandas index to deduplicate, then hands the frame to a Polars function for a fast join. `pl.from_pandas` drops the index. The join key is gone, so the analyst joins on row position with `hstack`. Two waves sorted differently, and 11% of respondents got another person's scores.

`pl.from_pandas(df.reset_index())` keeps the ID as a column, and the join uses it.

Worked example: weak code

```
1 big = pl.read_parquet("extracts/*.parquet").to_pandas()
2 big = big[big["wave"] == WAVE].set_index("respondent_id")
3 stats = pl.from_pandas(big).group_by("brand").agg(pl.col("weight").sum())
```

1. **Line 1.** Reads every wave eagerly, then copies all of it into pandas before filtering.
2. **Line 2.** The ID moves into the index.
3. **Line 3.** Back to Polars: the index, and so the ID, is dropped. Two conversions of the full data.

Worked example: strong code

```
 1 import pandas as pd
 2 import polars as pl
 3
 4
 5 def brand_totals(path_glob: str, wave: str) -> pd.DataFrame:
 6     """Weight per brand for one wave, as pandas for modelling. Grain: one
       row per brand."""
 7     return (
 8         pl.scan_parquet(path_glob)
 9         .filter(pl.col("wave") == wave)
10         .group_by("brand")
11         .agg(total=pl.col("weight").sum())
12         .sort("brand")
13         .collect()
```

```
(2)          .to_pandas()
15       )
```

1. **Line 9.** Reduce in the engine that reads the files.
2. **Line 14.** Convert once, at the end, when the result has one row per brand.

One primary engine. Reduce where the data is read. Convert small results at a clear boundary.

Recap card

- **Group-by over 10M rows: pandas, Polars, DuckDB** 0.22 s, 0.11 s, 0.06 s on 4 cores. All fast enough. Choose by size against memory and where data lives.
- **Mixing engines** Reduce first in the engine that reads the files, convert the small result last.
- **pl.from_pandas and the index** Dropped by default. reset_index() first, or include_index=True.
- **When Spark** Data already in the lakehouse, larger than one machine, or a scheduled production job.

Practice

Exercise 1. Multiple choice (level 1)

You have 40 GB of Parquet on a laptop with 16 GB of RAM. You need one table of weighted totals by wave and brand, once. Which is the best first choice?

- a) pandas read_parquet, then groupby
- b) DuckDB or Polars lazy, aggregating while reading
- c) Set up a Spark cluster
- d) Split the files by hand and run pandas on each

Why?

- r1) Both read Parquet in chunks and push filters and aggregation into the scan, so the full data never sits in memory.
- r2) They are newer than pandas.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0, Polars 1.33)

```
1 import pandas as pd
2 import polars as pl
3
```

```
pdf = pd.DataFrame({"respondent_id": ["r1", "r2"], "score": [3,
    4]}).set_index("respondent_id")
print(pl.from_pandas(pdf).columns)
```

Exercise 3. Fill the gap (level 2)

Hand a DuckDB result to Polars, then a Polars result to pandas.

```
top_brands = con.sql("SELECT brand, sum(weight) AS total FROM responses
    GROUP BY brand")._____(1)()
for_model = top_brands._____(2)()
```

Exercise 4. Spot the bug (level 3)

This function runs out of memory on the full extract. Tap the line that loads far more than the result needs, and fix the function.

What you need to know.

`df.item()` returns the single value of a Polars DataFrame that has exactly one row and one column, as a plain Python value. A query that ends in one aggregate, such as a mean, gives that shape:

```
pl.DataFrame({"x": [4.0]}).item()   # 4.0
```

```
import pandas as pd
import polars as pl


def wave_mean(path: str, wave: str) -> float:
    """Mean weight for one wave."""
    df = pl.read_parquet(path).to_pandas()
    return float(df.loc[df["wave"] == wave, "weight"].mean())
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 4)

Choose the primary engine for each job.

Options: pandas, Polars lazy, DuckDB, Spark.

1. Fit a logistic regression with statsmodels on a 50,000-row survey file
2. Ad hoc SQL joins across 30 CSV and Parquet exports on your laptop
3. Nightly job over tables that already live in the Databricks lakehouse
4. Heavy column transforms over 25 GB of Parquet on one 32 GB machine

Sources

- Group-by sum and mean over 10M rows: pandas 3.0.6 0.22 s, Polars 1.44.2 0.11 s, DuckDB 1.5.5 0.06 s (best of 3, 4 cores). (verified: Measured in this repository's dev environment.)

- pl.from_pandas drops the pandas index unless include_index=True. (verified: Polars API reference; run on Polars 1.33.1 and 1.44.2 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B4-L1 Polars expressions and lazy frames

B4-L1 exercise 1

Hints. (1) scan_ versus read_. (2) When does a LazyFrame run? (3) On collect().

Answer. b) Nothing is read yet. lf holds a query plan. Because: r1) A LazyFrame records steps and runs them only on collect().

`scan_parquet` builds a plan. `collect()` optimises and runs it, pushing the filter to the reader.

Common wrong answers.

- `a`: That is what pl.read_parquet followed by filter does. scan_parquet is lazy.
- `c`: Close, but no rows are read until collect(). Then only w9 rows are read.
- `d`: Nothing is written. A LazyFrame is a plan in memory.
- `b|r2`: Right answer, wrong reason. Polars reads Parquet itself.

B4-L1 exercise 2

Hints. (1) Which values are in group w2? (2) What is the sum of no non-null values? (3) Polars returns the additive identity.

Answer.

```
1 [3.0, 0.0]
```

The sum of an empty set of values is 0 in Polars. A real zero and "no data" look the same, so report `null_count()` alongside totals.

Common wrong answers.

- `[3.0, None]`: Polars sums an all-null group to 0, not null. Check null_count() when that matters.
- `[3.0]`: Every group appears in the output, including w2.

B4-L1 exercise 3

Hints. (1) A window over groups that keeps every row. (2) The method that runs a LazyFrame. (3) `over` and `collect`.

Answer.

```
out = (
    lf.with_columns(share=pl.col("weight") /
    pl.col("weight").sum().over("wave"))
    .collect()
)
```

`.over("wave")` broadcasts the wave total to each row. `collect()` runs the plan.

Common wrong answers.

- g1 = `group_by`: group_by collapses rows. over computes the group total and keeps every row.
- g2 = `execute`: A LazyFrame runs with collect().

B4-L1 exercise 4

Hints. (1) Aggregate to wave and brand first. (2) Get the wave total with `.sum().over("wave")`. (3) Guard with `pl.when(total > 0).then(...).otherwise(None)`.

Answer.

```
import polars as pl


def brand_share(df: pl.DataFrame) -> pl.DataFrame:
    """Weighted share of each brand within its wave. Grain: one row per wave and brand.

    Args:
        df: Columns wave, brand, weight.

    Returns:
        Columns wave, brand, share. share is null when the wave total is 0.
    """
    return (
        df.group_by("wave", "brand")
        .agg(brand_weight=pl.col("weight").sum())
        .with_columns(total=pl.col("brand_weight").sum().over("wave"))
        .with_columns(
            share=pl.when(pl.col("total") > 0)
            .then(pl.col("brand_weight") / pl.col("total"))
            .otherwise(None)
        )
        .select("wave", "brand", "share")
        .sort("wave", "brand")
    )
```

Aggregate to the output grain, broadcast the wave total with `over`, and guard the division so a zero total gives null instead of NaN.

B4-L1 exercise 5

Hints. (1) What does pl.len() count? (2) Does it count rows with a null score? (3) mean() skips nulls.

Answer. The bug is on line 8.

Fixed version:

```
 1 import polars as pl
 2
 3
 4 def mean_score(df: pl.DataFrame) -> pl.DataFrame:
 5     """Mean score per region among respondents who answered."""
 6     return (
 7         df.group_by("region")
 8         .agg(mean_score=pl.col("score").mean())
 9         .sort("region")
10     )
```

`pl.len()` counts every row, including skipped answers, so the denominator is too big. `pl.col("score").mean()` divides by the non-null count.

Common wrong answers.

- Line 7: Grouping by region is right. The formula inside agg is the problem.
- Line 9: Sorting does not change values.

B4-L2 DuckDB, SQL over files and DataFrames

B4-L2 exercise 1

Hints. (1) Which options build a new SQL string from the value? (2) Which one keeps the SQL fixed? (3) The placeholder.

Answer. b) `con.execute("... WHERE region = ?", [region])` Because: r1) The value is sent as data, separate from the SQL text.

A `?` placeholder keeps the SQL text fixed and passes the value as data.

Common wrong answers.

- `a`: An f-string pastes the value into the SQL. A quote in the value changes the query.
- `c`: Concatenation pastes the value into the SQL, the same risk as an f-string.
- `d`: % formatting is string pasting too, not a parameter.

- `b|r2`: Right answer, wrong reason. Safety comes from sending the value separately.

B4-L2 exercise 2

Hints. (1) How many rows does an aggregate without GROUP BY return? (2) What is SQL sum over no rows? (3) NULL, returned as None inside a tuple.

Answer.

```
(None,)
```

One row, holding NULL. Check `row[0] is None` before using the value.

Common wrong answers.

- `(0.0,)`: SQL sum over zero rows is NULL, which DuckDB returns as None.
- `None`: An aggregate without GROUP BY always returns one row. fetchone() gives a tuple holding None.

B4-L2 exercise 3

Hints. (1) The DuckDB table function for Parquet files. (2) The positional placeholder. (3) `read_parquet` and `?`.

Answer.

```
con.execute(
    "SELECT count(*) FROM read_parquet('extracts/*.parquet') WHERE wave
    = ?",
    [wave],
).fetchone()
```

`read_parquet` with a glob reads all files as one table. `?` binds the first value in the list.

Common wrong answers.

- g2 = `'{wave}'`: That is f-string pasting, and the string is not even an f-string here. Use a ? placeholder.
- g1 = `read_csv`: The files are Parquet. read_csv would fail to parse them.

B4-L2 exercise 4

Hints. (1) Replace the f-string with a ? placeholder and a list of values. (2) What does sum return when no rows match? (3) Return 0.0 when `row[0] is None`, else `float(row[0])`.

Answer.

```
import duckdb

REGION_TOTAL_SQL = "SELECT sum(weight) FROM responses WHERE region = ?"
```

```


def region_total(con: duckdb.DuckDBPyConnection, region: str) -> float:
    """Total weight for one region.

    Args:
        con: Connection holding a responses table.
        region: Region name, passed as a query parameter.

    Returns:
        The weight total, or 0.0 when no rows match.
    """
    row = con.execute(REGION_TOTAL_SQL, [region]).fetchone()
    if row is None or row[0] is None:
        return 0.0
    return float(row[0])
```

The placeholder keeps the SQL fixed, so quotes in the value are just characters. The None check turns SQL's NULL into the 0.0 the caller expects.

B4-L2 exercise 5

Hints. (1) Can a placeholder stand for a table name? (2) What stops arbitrary text becoming SQL? (3) Globals make testing hard (lesson B2-L1).

Answer.

- Values such as the wave go in as parameters.
- Identifiers such as table names cannot be parameters. Check them against an allowlist before building the SQL.
- Pass one connection in, so tests supply an in-memory database.

Parameters for values, an allowlist for identifiers, and a connection passed in.

Common wrong answers.

- wave = `fstring`: A quote in the wave changes the SQL. Use a parameter.
- table = `param`: Placeholders bind values, not identifiers. `FROM ?` is not valid SQL for a table name.
- table = `raw`: Any text the user types becomes SQL. Check it against the allowed tables first.
- con = `default`: The default connection is a shared global. Tests cannot swap in their own.

B4-L3 Choosing pandas, Polars, DuckDB or Spark

B4-L3 exercise 1

Hints. (1) Can the whole dataset sit in RAM? (2) Which tools aggregate while reading? (3) Scan-based engines on one machine.

Answer. b) DuckDB or Polars lazy, aggregating while reading Because: r1) Both read Parquet in chunks and push filters and aggregation into the scan, so the full data never sits in memory.

DuckDB and Polars lazy read in chunks and aggregate as they go. The result is small.

Common wrong answers.

- `a`: pandas loads the whole selection into memory first. 40 GB does not fit in 16 GB.
- `c`: A cluster works, but a one-off aggregate on one machine does not need one.
- `d`: Manual chunking re-implements what DuckDB and Polars do for you, with more room for mistakes.
- `b|r2`: Right answer, wrong reason. Age is not the point. Out-of-memory scanning is.

B4-L3 exercise 2

Hints. (1) Where is respondent_id after set_index? (2) Does Polars have an index? (3) The default is include_index=False.

Answer.

```
1 ['score']
```

The ID lives in the pandas index, and `from_pandas` drops the index by default. Call `reset_index()` first.

Common wrong answers.

- `['respondent_id', 'score']`: Polars has no index. from_pandas drops it unless you pass include_index=True.

B4-L3 exercise 3

Hints. (1) DuckDB relations have .df(), .pl() and .fetchall(). (2) Polars converts to pandas with a to_ method. (3) `pl` and `to_pandas`.

Answer.

```
top_brands = con.sql("SELECT brand, sum(weight) AS total FROM responses
    GROUP BY brand").pl()
for_model = top_brands.to_pandas()
```

`.pl()` returns a Polars DataFrame from DuckDB. `.to_pandas()` converts Polars to pandas through Arrow.

Common wrong answers.

- g1 = `df`: .df() returns pandas. The first step asks for Polars.
- g2 = `df`: Polars DataFrames convert with to_pandas().

B4-L3 exercise 4

Hints. (1) How much data exists after line 7? (2) Where should the filter run? (3) Scan, filter, aggregate, collect.

Answer. The bug is on line 7.

Fixed version:

```
import polars as pl


def wave_mean(path: str, wave: str) -> float:
    """Mean weight for one wave."""
    result = (
        pl.scan_parquet(path)
        .filter(pl.col("wave") == wave)
        .select(pl.col("weight").mean())
        .collect()
        .item()
    )
    return float(result)
```

Line 7 reads every wave eagerly, then copies it into pandas. A lazy scan pushes the wave filter into the reader and returns one number.

Common wrong answers.

- Line 8: The filter is correct, but by then every wave is already in memory twice.

B4-L3 exercise 5

Hints. (1) Where does the data live? (2) How big is it next to memory? (3) Which library does the next step need?

Answer.

- Small data feeding the Python modelling stack: pandas.
- SQL across many local files: DuckDB.
- Data already in the lakehouse, scheduled: Spark.

- Large local files with a transform pipeline: Polars lazy.

Size against memory, location of the data, and the next library in line decide the engine.

Common wrong answers.

- model = `spark`: 50,000 rows fit in memory many times over. Spark adds start-up time and a cluster for no gain.
- nightly = `pandas`: Pulling lakehouse tables into one pandas process moves the data away from the compute that already sits next to it.
- transform = `pandas`: pandas would load most of 25 GB into 32 GB of RAM. A lazy scan keeps memory in check.